

Parte 1: Funciones basicas

- GPIO
- ADC (Analog Digital Converter)
- PWM (Pulse Width Modulation)
- DAC (Digital to Analog Converter)
- Sensor Touch (Pines tactiles)
- UART echo (Universal Asynchronous Receiver-Transmitter)
- NVS (Non-Volatile Storage)
- Timers (GPTimer)
- Power Modes
- Utilidades del Sistema (Logging & Delays)

Desarrollo de temas

GPIO (General Purpose Input Output)

Libreria: `#include "driver/gpio.h"`

Estructura para configuracion

`gpio_config_t`

Variable	Descripcion	Tipo de variable
pin_bit_mask	Mascara del pin a setear (1ULL << Pin Deseado)	uint64_t
mode	modo del GPIO	gpio_mode_t
pull_up_en	Habilita la resistencia Pull-Up	gpio_pullup_t
pull_down_en	Habilita la resistencia Pull-Down	gpio pulldown_t
intr_type	El tipo de interrupcion	gpio_int_type_t

Aviso: Los pines 34, 35, 36 y 39 no tiene resistencia interna y solo son INPUT. Si intentas usarlos como OUTPUT o activar pull_up_en, no funcionará.

---> ==Caracteristicas==

1. Los modos que pueden tener el GPIO pueden ser, OUTPUT (Leds, actuadores, etc), INPUT (Pulsadores, sensores, etc), INPUT_OUTPUT (Habilita ambos para, por ejemplo, saber el estado del led), y los por ultimo, los modos open-drain (Utilizados para protocolos de comunicacion, por ejemplo, I2C). 1ULL significa un 1 en Unsigned Long Long, muy utilizado en aritmetica a nivel de bits. Si se quiere añadir un pin mas, se utiliza el signo OR a nivel de bits (" | "), ejemplo: (1ULL << PinDeseado0) | (1ULL << PinDeseado1).
2. Los valores que se le pueden dar a pullUp y pullDown pueden ser entre 0 o 1, no es necesario utilizar la macro.

3. Para los actuadores, el tipo de interrupcion suele ser DISABLE, pero suele ser muy utilizado por pulsadores, o entradas, para que interrumpa una tarea cuando se activan. Su uso se ve en el siguiente [ejemplo](#). Los valores de interrupt son demasiadas pero dependen principalmente de como quiero que se comporte la entrada; cuando la señal decae, es positiva, es negativa, etc.

Funcion de configuracion

```
esp_err_t gpio_config(const gpio_config_t *pGPIOConfig)
```

Algunos pines, como el GPIO12, GPIO13 pueden quedar en un estado desconocido tras el arranque. Esta función resetea el pin a su estado por defecto (deshabilitado) y limpia cualquier configuración previa del bootloader. Usar la funcion `gpio_reset_pin()`

---> Parametros

- pGPIOConfig: Puntero a la estructura de configuracion GPIO [in]

---> Retorna

- ESP_OK si no hay errores.
- ESP_ERR_INVALID_ARGS si hay errores de argumentos.

Funcion de seteo de salida

```
esp_err_t gpio_set_level(gpio_num_t gpio_num, uint32_t level)
```

---> Parametros

- gpio_num: Numero de GPIO. Si es GPIO16 por ejemplo, el parametro debe ser GPIO_NUM_16 (16) o bien puede ser el numero [in]
- level: Nivel de salida. 0 para LOW; 1 para HIGH [in]

---> Retorna

- ESP_OK si no hay errores.
- ESP_ERR_INVALID_ARGS si hay errores de argumentos.

Funcion entrada de señal

```
int gpio_get_level(gpio_num_t gpio_num)
```

AVISO: Si no fue configurado como entrada, el valor de retorno siempre es 0

---> Parametros

- gpio_num: Numero de GPIO. Si es GPIO16 por ejemplo, el parametro debe ser GPIO_NUM_16 (16) o bien puede ser el numero [in]

---> Retorna

- 0 si el nivel de la entrada es 0
- 1 si el nivel de la entrada es 1

ADC (Analog Digital Converter)

Libreria: `#include "esp_adc/adc_oneshot.h"`

Estructura para configuracion inicial ADC

`adc_oneshot_unit_init_cfg_t`

Variable	Descripcion	Tipo de variable
unit_id	Unidad del ADC	adc_unit_t
clk_src	Origen del reloj	adc_oneshot_clk_src_t
ulp_mode	ADC controlado por ULP	adc_ulp_mode_t

---> ==Caracteristicas==

- 1. La unidad del adc suele ser 1 (`ADC_UNIT_1`) o 2 (`ADC_UNIT_2`)
- 2. Selecciona el orien del reloj, comunmente se utiliza 0 para que utilice el default o por defecto
- 3. Selecciona si el adc va a funcionar cuando este en modo ULP. Comunmente se utiliza `ADC_ULP_MODE_DISABLE`. ULP (Ultra Low Power) procesador pequeño y dedicado que funciona con un consumo minimo de energia

Estructura para configuracion de canal

`adc_oneshot_chan_cfg_t`

Variable	Descripcion	Tipo de variable
atten	Atenuacion del ADC	adc_atten_t
bitwidth	Ancho de bits	adc_bitwidth_t

---> ==Caracteristicas==

- 1. Diferentes parametros determinan el rango del adc. Los valores posibles son, 0, 2.5, 6 y 12. Para señales debiles se usa `ADC_ATTEN_DB_0` y para sensores entandar `ADC_ATTEN_DB_12`
- 2. El ancho de bits del resultado de la conversión sin procesar. Un ejemplo es `ADC_BITWIDTH_DEFAULT`

Handle para ADC en modo oneshot

`adc_oneshot_unit_handle_t`

Funcion para configuracion inicial

```
esp_err_t adc_oneshot_new_unit(const adc_oneshot_unit_init_cfg_t *init_config,
adc_oneshot_unit_handle_t *ret_unit)
```

---> **Parametros**

- `init_config`: Puntero a la estructura de configuracion inicial [in]
- `ret_unit`: Puntero al Handler de la unidad [out]

--> **Retorna**

- `ESP_OK`: si no hay errores.
- `ESP_ERR_INVALID_ARG`: Argumentos invalidos
- `ESP_ERR_NO_MEM`: Memoria insuficiente
- `ESP_ERR_NOT_FOUND`: Periferico ADC en uso
- `ESP_FAIL`: El reloj no se inicializo correctamente

Funcion para configuracion de canal

```
esp_err_t adc_oneshot_config_channel(adc_oneshot_unit_handle_t handle, adc_channel_t channel, const adc_oneshot_chan_cfg_t *config)
```

AVISO: No utilizarla en un contexto ISR. La funcion es thread-safe.

---> **Parametros**

- `handle`: Handler de la unidad [in]
- `channel`: Canal ADC a configurar, su macro es `ADC_CHANNEL_0` y va desde 0 a 10 [in]
- `config`: Puntero a la estructura de configuracion de canal [in]

---> **Retorna**

- `ESP_OK`: si no hay errores
- `ESP_ERR_INVALID_ARG`: Argumentos invalidos

Funcion para leer el valor

```
esp_err_t adc_oneshot_read(adc_oneshot_unit_handle_t handle, adc_channel_t chan, int *out_raw)
```

---> **Parametros**

- `handle`: Handler de la unidad [in]
- `chan`: Canal ADC [in]
- `out_raw`: Puntero a la variable donde se guarda el valor leído [out]

---> **Retorna**

- `ESP_OK`: si no hay errores
- `ESP_ERR_INVALID_ARG`: Argumentos invalidos
- `ESP_ERR_TIMEOUT`: Timeout, el resultado del adc es invalido

Funcion para eliminar la unidad ADC

```
esp_err_t adc_oneshot_del_unit(adc_oneshot_unit_handle_t handle)
```

---> **Parametros**

- handle: Handler de la unidad [in]

---> Retorna

- ESP_OK: si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos
- ESP_ERR_NOT_FOUND: El periferico no se utiliza.

==Funcion complementaria==

Funcion para saber el canal a partir del numero de GPIO

```
esp_err_t adc_oneshot_io_to_channel(int io_num, adc_unit_t *const unit_id, adc_channel_t *const channel)
```

---> Parametros

- io_num: Numero de GPIO [in]
- unit_id: Unidad ADC [out]
- channel: Canal ADC [out]

---> Retorna

- ESP_OK: si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos
- ESP_ERR_NOT_FOUND: El pin no es un pad ADC valido

==Calibracion del ADC==

Libreria: `#include "esp_adc/adc_cali.h"` y `#include "esp_adc/adc_cali_scheme.h"`

Estructura para la configuracion de calibracion lineal

```
adc_cali_line_fitting_config_t
```

Variable	Descripcion	Tipo de variable
unit_id	La unidad ADC en donde vienen los valores	adc_unit_t
atten	Atenuacion que el ADC utiliza	adc_bitwidth_t
bitwidth	Ancho de bits que el ADC utiliza	adc_bitwidth_t
default_vref	Normalmente es 0, utilizado en caso de usar eFuse	adc_cali_line_fitting_efuse_val_t

Handle para la calibracion: `adc_cali_handle_t`

Funcion para configuracion de calibracion lineal

```
esp_err_t adc_cali_create_scheme_line_fitting(const adc_cali_line_fitting_config_t *config, adc_cali_handle_t *ret_handle)
```

---> Parametros

- config: Puntero a la estructura de configuracion de calibracion lineal [in]
- ret_handle: Puntero al handle de calibracion [out]

---> Retorna

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos
- ESP_ERR_NO_MEM: Memoria insuficiente
- ESP_ERR_NOT_SUPPORTED: Error con eFuse

Funcion para convertir el valor ADC a milivoltios

```
esp_err_t adc_cali_raw_to_voltage(adc_cali_handle_t handle, int raw, int *voltage)
```

---> Parametros

- handle: Handle de calibracion [in]
- raw: Dato crudo obtenido por la funcion `adc_oneshot_read` [in]
- voltaje: Puntero a la variable donde se guarda el valor calibrado, en mV [out]

---> Retorna

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos
- ESP_ERR_INVALID_STATE: Estado invalido, *scheme* no registrado

Funcion para eliminar la unidad calibracion ADC

```
esp_err_t adc_cali_delete_scheme_line_fitting(adc_cali_handle_t handle)
```

---> Parametros

- handle: Handle de calibracion [in]

---> Retorna

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos

ADC Continuous Mode (Lectura por DMA)

Nota para usuarios avanzados (DSP / Audio): Si el proyecto requiere leer señales a alta velocidad (muestreo constante) para procesar audio, calcular FFT (Transformada Rápida de Fourier) o analizar vibraciones, NO uses el modo OneShot.

¿Por qué?: El modo OneShot es bloqueante y lento para estas tareas.

La Solución: Utiliza el Continuous Mode. Este modo utiliza el controlador DMA (Direct Memory Access) para llenar un buffer de datos automáticamente en segundo plano sin ocupar la CPU.

Librería: `#include "esp_adc/adc_continuous.h"`

Concepto clave: Configuras una frecuencia de muestreo (ej. 44100 Hz) y un patrón de canales. El hardware te avisa mediante una interrupción cuando el buffer está lleno y listo para procesar.

PWM (Pulse Widht Modulation)

Libreria: `#include "driver/ledc.h"`

Estructura para configuracion de timers LEDC

`ledc_timer_config_t`

Variable	Descripcion	Tipo de variable
speed_mode	Velocidad del LEDC, puede ser high speed o low speed	ledc_mode_t
duty_resolution	Resolucion del canal	ledc_timer_bit_t
timer_num	Origen del timer del canal	ledc_timer_t
freq_hz	Frecuencia del timer LEDC en Hz	uint32_t
clk_cfg	Origen del reloj	ledc_clk_cfg_t
deconfigure	Desconfigura el LEDC	bool

---> ==Caracteristicas==

1. Velocidad del LEDC, puede ser high `LEDC_HIGH_SPEED_MODE`, low `LEDC_LOW_SPEED_MODE` o el limite `LEDC_SPEED_MODE_MAX`. Se recomienda low speed
2. Resolucion del canal, va desde `LEDC_TIMER_1_BIT` hasta `LEDC_TIMER_20_BIT`. Para Dimming se utiliza 13 bits. Para servomotores de 10 a 14 bits. Para buzzers el valor es fijo, como 10 bits
3. Timer, va desde `LEDC_TIMER_0` hasta `LEDC_TIMER_3`.
4. Frecuecia recomendada: de 4000 a 5000 para Dimming de LEDs. Servomotores, 50. Rara Buzzers puede variar.
5. Estructura con la configuracion del origen del reloj
6. Vincula el temporizador configurado previamente con este canal específico

Funcion para configuracion de timers LEDC

`esp_err_t ledc_timer_config(const ledc_timer_config_t *timer_conf)`

---> **Parametros**

- timer_conf: Puntero a la estructura con la configuracion de timers [in]

---> **Retorna**

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos
- ESP_FAIL: No se pudo encontrar un numero base a partir de la frecuencua y resolucion dada

- ESP_ERR_INVALID_STATE: El timer no puede ser desconfigurado debido a que no fue configurado o no fue pausado

Estructura para configuracion de canal LEDC

ledc_channel_config_t

Variable	Descripcion	Tipo de variable
gpio_num	Numero de GPIO a utilizar	int
speed_mode	Velocidad del LEDC, puede ser high speed o low speed	ledc_mode_t
channel	Canal LEDC	ledc_channel_t
intr_type	Configura la interrupcion, habilitado o desabilitado	ledc_intr_type_t
timer_sel	Origen del timer del canal	ledc_timer_t
duty	duty del canal	uint32_t
hpoint	valor del hpoint del LEDC	int
sleep_mode	Actitud del LEDC en modo light-sleep	ledc_sleep_mode_t
output_invert	Habilita o desabilita la salida GPIO invertida	unsigned int
flags	Flags del LEDC	ledc_channel_config_t

---> ==Caracteristicas==

1. Utilizar los pines GPIO con salida PWM, en un ESP32, la mayoría de sus pines tienen esta salida
2. Misma velocidad ya seteada en la configuracion anterior
3. Seleccionar canal, desde LEDC_CHANNEL_0 hasta LEDC_CHANNEL_7, los pines seteados con el mismo canal actuan de la misma manera
4. Lo mismo visto con entradas digitales, se pueden configurar interrupciones, depeden el caso pero por ahora usaremos LEDC_INTR_DISABLE
5. Mismo timer ya seteado en la configuracion anterior
6. Representa la cantidad de tiempo encendido. Va desde 0 hasta (2**duty_resolution)
7. Define el desfase de la señal PWM. Normalmente es 0 o hasta (2**duty_resolution)-1
8. Normalmente se utiliza LEDC_SLEEP_MODE_NO_ALIVE_NO_PD que significa que no hay salida PWM
9. La salida GPIO invertida significa que un led se enciende cuando se le aplica 0. Dependiendo el caso es 0 o 1
10. Normalmente es 0

Funcion para configuracion de canal LEDC

esp_err_t ledc_channel_config(const ledc_channel_config_t *ledc_conf)

---> **Parametros**

- ledc_conf: Puntero a la estructura de configuracion de canal

---> **Retorna**

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos

Funcion para setear el estado de los LEDC

```
esp_err_t ledc_set_duty(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t duty)
```

---> Parametros

- speed_mode: Velocidad antes configurada
- channel: Canal en donde se desea setear
- duty: Cantidad de tiempo encendido. Va desde 0 hasta (2**duty_resolution)

---> Retorna

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos

Funcion para actualizar el estado de los LEDC

```
esp_err_t ledc_update_duty(ledc_mode_t speed_mode, ledc_channel_t channel)
```

---> Parametros

- speed_mode: Velocidad antes configurada
- channel: Canal en donde se desea actualizar

---> Retorna

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos

PWM en un ambiente multitareas

Si se desea aplicar estas funciones en un ambiente multithreading o multitareas, se recomienda la siguiente

funcion: `esp_err_t ledc_set_duty_and_update(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t duty, uint32_t hpoint)`

---> Parametros

- speed_mode: Velocidad antes configurada
- channel: Canal al que se desea aplicar el cambio
- duty: Cantidad de tiempo encendido. Va desde 0 hasta (2**duty_resolution)
- hpoint: Desfaze del LEDC. Va desde 0 hasta (2**duty_resolution) - 1

---> Retorna

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_STATE: Canal no inicializado
- ESP_ERR_INVALID_ARG: Argumentos invalidos
- ESP_FAIL: Inicio de funcion fade dio error

Funcionalidad: Hardware Fading (Desvanecimiento)

Permite cambiar el ciclo de trabajo (duty) de un valor a otro progresivamente en un tiempo determinado, gestionado automáticamente por el hardware.

Funcion para instalar el modo fading

```
esp_err_t ledc_fade_func_install(int intr_alloc_flags)
```

---> Parametros

- Flags: usa 0 por defecto

---> Retorna

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Error de flag
- ESP_ERR_NOT_FOUND: Fallo al intentar buscar fuente de interrupcion disponible
- ESP_ERR_INVALID_STATE: Funcion fade ya instalada

Funcion para desisntalar modo fade

```
void ledc_fade_func_uninstall(void)
```

Funcion para configurar el desvanecimiento

```
esp_err_t ledc_set_fade_with_time(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t target_duty, int desired_fade_time_ms)
```

---> Parametros

- speed_mode: Velocidad antes configurada
- channel: Canal al que se desea aplicar el cambio
- target_duty: Cantidad de tiempo objetivo. Va desde 0 hasta (2**duty_resolution)
- desired_fade_time_ms: Tiempo deseado de fading en milisegundos

---> Retorna

- ESP_OK: Si no hay errores
- ESP_ERR_INVALID_ARG: Argumentos invalidos
- ESP_ERR_INVALID_STATE: Canal no inicializado
- ESP_FAIL: Inicio de funcion fade dio error

Funcion para inicar el modo fade

```
esp_err_t ledc_fade_start(ledc_mode_t speed_mode, ledc_channel_t channel, ledc_fade_mode_t fade_mode)
```

---> Parametros

- `speed_mode`: Velocidad antes configurada
- `channel`: Canal al que se desea aplicar el cambio
- `fade_mode`: Tiene dos opciones. `LEDC_FADE_NO_WAIT` para que la función retorne inmediatamente y el LED cambia de brillo en segundo plano. Esta es la mas recomendada. `LEDC_FADE_WAIT_DONE` para que la función congele tu tarea hasta que el LED termine de cambiar

---> Retorna

- `ESP_OK`: Si no hay errores
- `ESP_ERR_INVALID_STATE`: Canal no inicializado o modo fade no instalado
- `ESP_ERR_INVALID_ARG`: Argumentos invalidos

Recomendaciones: No llamar a `ledc_set_duty` mientras el fade esta en uso. Si se desea hacer un fade muy largo, reducir la frecuencia. Existe una estructura avanzada llamada `ledc_fade_param_config_t`, pero se usa solo para encadenar múltiples desvanecimientos complejos (curvas gamma).

Aplicaciones

Efecto "Respiración (Breathing LED): Indicadores de estado en dispositivos IoT (ej: "Conectando a WiFi..."). Se ve mucho más profesional que un parpadeo seco.

Arranque Suave de Motores (Soft-Start): Si usas PWM para un motor DC, cambiar de 0% a 100% de golpe genera un pico de corriente enorme. Usar un fade de 500ms reduce el estrés eléctrico y mecánico.

Domótica (Iluminación): Al encender/apagar luces de una habitación, una transición de 200-400ms es mucho más agradable a la vista que un encendido instantáneo.

DAC (Digital to Analog Converter)

Libreria: `#include "driver/dac_oneshot.h"`

Estructura para configurar DAC en modo oneshot

`dac_oneshot_config_t`

Variable	Descripcion	Tipo de variable
<code>chan_id</code>	Canal DAC	<code>dac_channel_t</code>

---> ==Caracteristicas==

1. El canal debe ser `DAC_CHAN_0` que es el GPIO25 o `DAC_CHAN_1` que es el GPIO26

Handler para controlar el DAC en modo oneshot

`dac_oneshot_handle_t`

Funcion para configurar el DAC en modo oneshot

```
esp_err_t dac_oneshot_new_channel(const dac_oneshot_config_t *oneshot_cfg,
dac_oneshot_handle_t *ret_handle)
```

---> **Parametros**

- oneshot_cfg: Puntero a la estructura con la configuracion oneshot [in]
- ret_handle: Puntero al handle para el DAC oneshot [out]

---> **Retorna**

- ESP_ERR_INVALID_ARG: El parametro de entrada es invalido
- ESP_ERR_INVALID_STATE: El canal DAC ya fue registrado
- ESP_ERR_NO_MEM: Memoria insuficiente
- ESP_OK: Se pudo localizar el canal DAC. Salio todo bien.

Funcion para eliminar el DAC

```
esp_err_t dac_oneshot_del_channel(dac_oneshot_handle_t handle)
```

---> **Parametros**

- handle: handle del DAC oneshot [in]

---> **Retorna**

- ESP_ERR_INVALID_ARG: El parametro de entrada es invalido
- ESP_ERR_INVALID_STATE: El canal ya fue eliminado
- ESP_OK: Se pudo eliminar el canal. Salio todo bien.

Funcion para escribir el voltaje

```
esp_err_t dac_oneshot_output_voltage(dac_oneshot_handle_t handle, uint8_t digi_value)
```

---> **Parametros**

- handle: handle del DAC oneshot [in]
- digi_value: Valor digital que sera convertido. Los valores van desde 0 hasta 255 [in]

---> **Retorna**

- ESP_ERR_INVALID_ARG: El parametro de entrada es invalido
- ESP_OK: La conversion digital fue exitosa

Librerias adicionales

El dispositivo DAC tambien se lo puede configurar para que tenga una salida *cosenoidal* y otra para que salga de manera *continua*

En la libreria `#include "driver/dac_cosine.h"` se encuentra todo lo referido para que el DAC tenga forma cosenoidal. Aqui se puede configurar el reloj, frecuencia, offset, atenuacion, fase, etc.

En la libreria `#include "driver/dac_continuous.h"` se encuentra todo lo referido para que el DAC tenga salida continua. Un uso es para generar señales de audio o formas de onda de forma continua, a menudo usando el controlador I2S para DMA y un búfer cíclico para reproducir audio sin interrupciones, como tonos, música o patrones, ideal para aplicaciones de sonido, control de motores y generación de señales de prueba.

Sensor touch (Pines tactiles)

Libreria: `#include "driver/touch_sens.h"`

Para un ESP32 se recomienda usar `driver/touch_pad.h`, por lo que a la hora de utilizar la libreria, verificar uso

Estructura para configurar el sensor touch

`touch_sensor_config_t`

Variable	Descripcion	Tipo de variable
power_on_wait_us	El tiempo de espera entre el encendido de canal y la medicion	uint32_t
meas_interval_us	Intervalo de medicion entre cada canal	uint32_t
intr_trig_mode	Tipo de disparo del modo interrupcion	touch_intr_trig_mode_t
intr_trig_group	Los grupos de canales que "disparan"	touch_intr_trig_group_t
sample_cfg_num	El numero de configuracion de muestra	uint32_t
sample_cfg	Array a la configuracion de muestra	touch_sensor_sample_config_t*

---> ==Caracteristicas==

1. Valor recomendado: 0. El hardware gestiona bien el encendido. Aumentar solo si el sensor es inestable al arrancar.
2. Valor recomendado: 10000 a 20000. Define que tan rapido reacciona. <10ms: Muy rapido, alto consumo. >50ms: Lento al tocar, bajo consumo
3. Debido a que con el ESP32 V1 el valor baja cuando se presiona, hay que utilizar:
`TOUCH_INTR_TRIG_ON_BELOW_THRESH`
4. Para uso estandar usar `TOUCH_INTR_TRIG_GROUP_NORMAL`
5. El ESP32 original solo soporta una configuración de muestreo a la vez. Utilizar numero 1.
6. Es el puntero a estructura `touch_sensor_sample_config_t` que es la configuracion de muestra. Al crear el array de tamaño `sample_cfg_num`, su unico valor es lo que devuelve la macro `TOUCH_SENSOR_V1_DEFAULT_SAMPLE_CONFIG()`, esto configura automáticamente voltajes de referencia y divisores de reloj estables.

Handler para controlar el sensor touch

`touch_sensor_handle_t`

Funcion para configurar el sensor touch

```
esp_err_t touch_sensor_new_controller(const touch_sensor_config_t *sens_cfg,
touch_sensor_handle_t *ret_sens_handle)
```

---> Parametros

- sens_cfg: Puntero a la estructura con la configuracion del sensor touch [in]
- ret_sens_handle: Puntero al handle para configurar el sensor touch [out]

---> Retona

- ESP_OK: Si salio todo bien
- ESP_ERR_NO_MEM: Memoria insuficiente para el controlador
- ESP_ERR_INVALID_ARG: Argumentos invalidos o puntero a NULL
- ESP_ERR_INVALID_STATE: El controlador touch ya se uso

Funcion para eliminar el controlador del sensor touch

```
esp_err_t touch_sensor_del_controller(touch_sensor_handle_t sens_handle)
```

---> Parametros

- sens_handle: Handle del sensor touch

---> Retona

- ESP_OK: Si todo salio bien
- ESP_ERR_INVALID_ARG: Argumentos invalidos o puntero a NULL
- ESP_ERR_INVALID_STATE: Controlador no desabilitado o algunos canales no fueron eliminados

Estructura a la configuracion de canal del sensor touch

```
touch_channel_config_t
```

Variable	Descripcion	Tipo de variable
abs_active_thresh	El umbral de activacion	uint32_t
charge_speed	Velocidad de carga y descarga del pad touch	touch_charge_speed_t
init_charge_volt	Voltaje inicial antes de cargar y descargar el pad tactil	touch_init_charge_volt_t
group	El grupo en la que el canal pertenece. Es usado para disparar el interruptor	touch_chan_trig_group_t

---> ==Caracteristicas==

1. Es recomendable entre 1000 y 3000 dependiendo del material que cubre el sensor.
2. Se recomienda usar el default, `TOUCH_CHARGE_SPEED_DEFAULT`. Bajarle el valor reduce el ruido pero baja sensibilidad
3. Se recomienda `TOUCH_INIT_CHARGE_VOLT_DEFAULT` que generalmente es 2,7. Un voltaje más alto otorga mayor rango dinámico (la señal tiene más "espacio" para moverse antes de saturarse),

mejorando la sensibilidad y la resolución.

4. Usar `TOUCH_CHAN_TRIG_GROUP_NORMAL`. Esto asegura que el canal participe en la lógica de interrupciones estándar.

Handle para la configuracion de canal

`touch_channel_handle_t`

Funcion para registrar un nuevo sensor touch a partir del controlador

```
esp_err_t touch_sensor_new_channel(touch_sensor_handle_t sens_handle, int chan_id, const touch_channel_config_t *chan_cfg, touch_channel_handle_t *ret_chan_handle)
```

---> Parametros

- `sens_handle`: Handle del controlador sensor touch. Es el handle antes utilizado[in]
- `chan_id`: Canal touch, va desde el 0 hasta el 9 [in]
- `chan_cfg`: Puntero onfiguracion del canal touch [in]
- `ret_chan_handle`: Puntero al handle de la configuracion de canal [out]

---> Retorna

- `ESP_OK`: Si salio todo bien
- `ESP_ERR_NO_MEM`: Memoria insuficiente para el canal del sensor touch
- `ESP_ERR_INVALID_ARG`: Argumentos invalidos o puntero a NULL
- `ESP_ERR_INVALID_STATE`: El controlador al sensor touch no esta habilitado o canal ya usado

Funcion para habilitar el sensor touch

```
esp_err_t touch_sensor_enable(touch_sensor_handle_t sens_handle)
```

---> Parametros

- `sens_handle`: Handle del controlador del sensor touch [in]

---> Retorna

- `ESP_OK`: Si salio todo bien
- `ESP_ERR_INVALID_ARG`: Argumentos invalidos o puntero a NULL
- `ESP_ERR_INVALID_STATE`: El controlador del sensor touch ya fue habilitado

Funcion para leer el valor del pin

```
esp_err_t touch_channel_read_data(touch_channel_handle_t chan_handle, touch_chan_data_type_t type, uint32_t *data)
```

---> Parametros

- `chan_handle`: Handle de canal touch [in]
- `type`: Especifica el tipo de dato a leer. Se utiliza `TOUCH_CHAN_DATA_TYPE_RAW` para leer el valor crudo [in]

- data: Puntero al array de datos [out]
- > **Retorna**
- ESP_OK: Si salio todo bien
 - ESP_ERR_INVALID_ARG: Argumentos invalidos o puntero a NULL
-

UART (Universal Asynchronous Receiver-Transmitter)

Libreria: `#include "driver/uart.h"`

Funcion para instalar el driver UART

```
esp_err_t uart_driver_install(uart_port_t uart_num, int rx_buffer_size, int tx_buffer_size, int queue_size, QueueHandle_t *uart_queue, int intr_alloc_flags)
```

- > **Parametros**
- uart_num: Numero de puerto UART. Puede ser `UART_NUM_0` hasta `UART_NUM_2`
 - rx_buffer_size: Tamaño del buffer del receptor. Un valor por ejemplo es `1024 * 2`
 - tx_buffer_size: Tamaño del buffer del transmisor. Para empezar, usar siempre 0 en TX buffer
 - queue_size: Tamaño de la cola, este no guarda datos sino eventos. Una cola de 10 a 20 es lo mas usado
 - uart_queue: Puntero al handle de eventos FreeRTOS. Este tema se vera mas adelante. Si solo va a leer por polling colocar NULL
 - intr_alloc_flags: Flags usandos para la interrupcion. Comunmete se usa 0
- > **Retorna**
- ESP_OK: Si salio todo bien
 - ESP_FAIL: Error de parametros

Estructura con la configuracion UART

`uart_config_t`

Variable	Descripcion	Tipo de variable
baud_rate	Los baudios de comunicacion	int
data_bits	Tamaño del byte UART	uart_word_length_t
parity	Paridad del modo UART	uart_parity_t
stop_bits	Bits de stop	uart_stop_bits_t
flow_ctrl	Modo control de flujo	uart_hw_flowcontrol_t
rx_flow_ctrl_thresh	Umbral RTS HS	uint8_t
source_clk	Seleccion del reloj de origen	uart_sclk_t
allow_pd	Guarda los registros en modo sleep	uint32_t

Variable	Descripcion	Tipo de variable
flags	Estructura con las flags de configuracion	int

---> ==Caracteristicas==

1. Para un ESP32 se suele utilizar 115200 baudios. Un valor como 9600 se utilizaba para modelos GPS viejos. Para una alta velocidad se utiliza 921600
2. En la mayoría de casos se usa `UART_DATA_8_BITS`
3. En la mayoría de casos se usa `UART_PARITY_DISABLE`
4. En la mayoría de casos se usa `UART_STOP_BITS_1`
5. Si se usan 2 cables usar `UART_HW_FLOWCTRL_DISABLE`. Si se usan 4 cables usar `UART_HW_FLOWCTRL_CTS_RTS`, vital para altas velocidades (> 1Mbps) para evitar que el buffer se desborde si el ESP32 no alcanza a leer
6. Con `UART_SCLK_DEFAULT` deja que el driver elija, normalmente usa el reloj APB de 80MHz

Funcion para configurar los aspectos de comunicacion UART

```
esp_err_t uart_param_config(uart_port_t uart_num, const uart_config_t *uart_config)
```

---> **Parametros**

- `uart_num`: Numero de puerto UART [in]
- `uart_config`: Puntero a la estructura con la configuracion UART [in]

---> **Retorna**

- `ESP_OK`: Si salio todo bien -`ESP_FAIL`: Error de parametros o baudios inalcanzable

Funcion para configurar los pines de comunicacion

```
esp_err_t uart_set_pin(uart_port_t uart_num, int tx_io_num, int rx_io_num, int rts_io_num, int cts_io_num)
```

Utilizar la macro `UART_PIN_NO_CHANGE` sobre un paramtros indica que no se va a usar el pin

---> **Parametros**

- `uart_num`: Numero de puerto UART
- `tx_io_num`: Numero de pin GPIO para Tx
- `rx_io_num`: Numero de pin GPIO para Rx
- `rts_io_num`: Numero de pin GPIO para Rts
- `cts_io_num`: Numero de pin GPIO para Cts

---> **Retorna**

- `ESP_OK`: Si salio todo bien
- `ESP_FAIL`: Error de parametros

Funcion para enviar datos UART

```
int uart_write_bytes(uart_port_t uart_num, const void *src, size_t size)
```

---> **Parametros**

- uart_num: Numero de puerto UART
- src: Puntero al buffer con datos
- size: Tamaño de los datos a enviar

---> **Retorna**

- -1 si hubo error en los parametros
- Otros, que son valores a partir de 0, siendo el numero de bytes yendo al FIFO Tx. FIFO; First Input first output. Nomenclatura que referencia que el primer dato que entra es el primero en salir

Funcion para recibir o leer datos UART

```
int uart_read_bytes(uart_port_t uart_num, void *buf, uint32_t length, TickType_t ticks_to_wait)
```

---> **Parametros**

- uart_num: Numero de puerto UART
- buf: Puntero al buffer donde se guardan los datos
- lenght: Tamaño de los datos
- ticks_to_wait: sTimeout o espera, contado en RTOS Ticks. Utilizar la macro pdMS_TO_TICKS para pasar de milisegundos a ticks

---> **Retorna**

- -1 si hubo error en los parametros
- Otros, que son valores a partir de 0, siendo el numero de bytes leidos

Estas dos funciones son las principales, despues hay algunas que son complementarias que sirven para ciertas situaciones, como por ejemplo: para limpiar el Buffer de entrada `uart_flush_input()`, consultar datos disponibles `uart_get_buffered_data_len()` y para esperar que se termine el envio `uart_wait_tx_done`

NVS (Non Volatile Storage)

Libreria: `#include "nvs_flash.h"` y `#include "nvs.h"`

Funcion para inicializar el NVS

```
esp_err_t nvs_flash_init(void)
```

---> **Retorna**

- ESP_OK: Si el almacenamiento fue inicializado correctamente
- ESP_ERR_NVS_NO_FREE_PAGES: Si el almacenamiento NVS no contiene paginas vacias (Puede suceder si la particion nvs fue truncada)

- ESP_ERR_NOT_FOUND: Si no hay particion con la etiqueta "nvs" es encontrada en la tabla de particiones
- ESP_ERR_NO_MEM: En caso de que la memoria no fue localizada en las estructuras internas
- Uno de los codigos de error del "subyacente" driver de almacenamiento
- Codigos de error de `nvs_flash_read_security_cfg` API (cuando "NVS_ENCRYPTION" este habilitado).
- Codigos de error de `nvs_flash_generate_keys` API (cuando "NVS_ENCRYPTION" este habilitado).
- Codigos de error de `nvs_flash_secure_init_partition` API (cuando "NVS_ENCRYPTION" este habilitado)

AVISO: En caso de que el error sea `NO_FREE_PAGES` O `NEW_VERSION_FOUND` se debe llamar a la funcion `nvs_flash_erase()` para que borre la particion NVS

Handle para manejar los NVS

`nvs_handle_t`

Funcion para la apertura del nvs

```
esp_err_t nvs_open(const char *namespace_name, nvs_open_mode_t open_mode, nvs_handle_t *out_handle)
```

---> Parametros

- namespace_name: Puntero al nombre del espacio de trabajo. La maxima cantidad de caracteres es de 16
- open_mode: Modo de apertura. Puede ser `NVS_READONLY` para unicamente leer o `NVS_READWRITE` para leer y escribir
- out_handle: Puntero al handle manejador de NVS

---> Retorna

- ESP_OK: si el controlador de almacenamiento se abrió correctamente
- ESP_FAIL: si hay un error interno; Probablemente debido a una partición NVS dañada (solo si las comprobaciones de aserción NVS están deshabilitadas).
- ESP_ERR_NVS_NOT_INITIALIZED: si el controlador de almacenamiento no está inicializado.
- ESP_ERR_NVS_PART_NOT_FOUND: si no se encuentra la partición con la etiqueta "nvs".
- ESP_ERR_NVS_NOT_FOUND: si el espacio de nombres aún no existe y el modo es NVS_READONLY.
- ESP_ERR_NVS_INVALID_NAME: si el nombre del espacio de nombres no cumple las restricciones.
- ESP_ERR_NO_MEM: si no se pudo asignar memoria para las estructuras internas.
- ESP_ERR_NVS_NOT_ENOUGH_SPACE: si no hay espacio para una nueva entrada o hay demasiados espacios de nombres diferentes (máximo permitido de espacios de nombres diferentes: 254).
- ESP_ERR_NOT_ALLOWED: si la partición NVS es de solo lectura y el modo es NVS_READWRITE
- ESP_ERR_INVALID_ARG: si el identificador de salida es NULL
- Otros códigos de error del controlador de almacenamiento subyacente

Funcion para cerrar el modo nvs

```
void nvs_close(nvs_handle_t handle)
```

---> Parametros

- handle: Handle del NVS

Funcion para setear el dato

```
esp_err_t nvs_set_xn(nvs_handle_t handle, const char *key, xintn_t value)
```

x: Usar "u" si el dato es unsigned. Usar "i" si es con signo. n: Tipo de dato, ya sea 8, 16, 32 o 64 Otros tipos: str para string. blob para una estructura. Para ambos casos hay que especificar el tamaño

---> Parametros

- handle: Handle del NVS
- key: Puntero al nombre o key a setear
- value: Valor a setear
- lenght: En caso de que la funcion sea str o blob. Tamaño del dato en bytes

---> Retorna

- ESP_OK si el valor se configuró correctamente
- ESP_FAIL si hay un error interno; probablemente debido a una partición NVS dañada (solo si las comprobaciones de aserción NVS están deshabilitadas)
- ESP_ERR_NVS_INVALID_HANDLE si el controlador se ha cerrado o es nulo
- ESP_ERR_NVS_READ_ONLY si el controlador de almacenamiento se abrió como de solo lectura
- ESP_ERR_NVS_INVALID_NAME si el nombre de la clave no cumple las restricciones
- ESP_ERR_NVS_NOT_ENOUGH_SPACE si no hay suficiente espacio en el almacenamiento subyacente para guardar el valor
- ESP_ERR_NVS_REMOVE_FAILED si el valor no se actualizó debido a un error en la escritura en la memoria flash. Sin embargo, el valor se escribió y la actualización finalizará tras la reinicialización de NVS, siempre que la operación de escritura no vuelva a fallar.
- ESP_ERR_NVS_VALUE_TOO_LONG si el valor es demasiado largo

Funcion para escribir los cambios

```
esp_err_t nvs_commit(nvs_handle_t handle)
```

---> Parametros

- handle: Handle del NVS

---> Retorna

- ESP_OK: Si los cambios fueron escritos correctamente
- ESP_ERR_NVS_INVALID_HANDLE: Si el handle fue eliminado o es NULL
- Otros errores del driver de almacenamiento

Funcion para obtener un dato

```
esp_err_t nvs_get_xn(nvs_handle_t handle, const char *key, xintn_t *out_value)
```

x: Usar "u" si el dato es unsigned. Usar "i" si es con signo. n: Tipo de dato, ya sea 8, 16, 32 o 64 Otros tipos: str para string. blob para una estructura. Para ambos casos hay que especificar el tamaño.
¡IMPORTANTE! El nombre de la clave (key) tiene un límite máximo de 15 caracteres (sin contar el terminador nulo)

- handle: Handle del NVS
- key: Puntero al nombre o key a buscar
- out_value: Puntero al valor de salida
- lenght: En caso de que la funcion sea str o blob. Puntero del tamaño del dato en bytes

---> **Retorna**

- ESP_OK si el valor se recuperó correctamente
- ESP_FAIL si hay un error interno; probablemente debido a una partición NVS dañada (solo si las comprobaciones de aserción NVS están deshabilitadas)
- ESP_ERR_NVS_NOT_FOUND si la clave solicitada no existe
- ESP_ERR_NVS_INVALID_HANDLE si el identificador se ha cerrado o es nulo
- ESP_ERR_NVS_INVALID_NAME si el nombre de la clave no cumple las restricciones
- ESP_ERR_NVS_INVALID_LENGTH si la longitud no es suficiente para almacenar los datos

GPTimer (General Purpose Timer)

Libreria: `#include "driver/gptimer.h"`

Estructura de configuracion del GPT

`gptimer_config_t`

Variable	Descripcion	Tipo de variable
clk_src	Origen del reloj GPT	gptimer_clock_source_t
direction	Direccion de cuenta	gptimer_count_direction_t
resolution_hz	Resolucion de contado o frecuencia de trabajo en Hz	uint32_t
intr_priority	Prioridad de interrupcion	int
intr_shared	Comparte el timer interruptor con otro perifericos	uint32_t
allow_pd	Permite al driver que la energia sea apagada cuando entre en modo sleep	uint32_t
flags	Flags de configuracion GPT	0

---> ==Caracteristicas==

1. Se utiliza `GPTIMER_CLK_SRC_DEFAULT`, normalmente deriva del reloj APB a 80MHz
2. Setear la direccion de contado es, ya sea para arriba o para abajo. Se usa `GPTIMER_COUNT_UP` o `GPTIMER_COUNT_DOWN`

3. Resolucion del contador interno. Cada tick es equivalente a $(1 / \text{resolucion})$ segundos. Un valor de frecuencia utilizado es 1 MHz o sea (1000×1000) Hz, que usando la formula, seria un tick es un microsegundo
4. Setea la prioridad de interrupcion. Dependiendo de su uso, va de 0 (Prioridad por defecto) a 3 (Maxima prioridad)
5. Esta seccion le permite al driver hacer un backup en la memoria cuando entre en modo sleep. Segun la documentacion, puede consumir hasta 30 bytes.
6. Cambios leves en algunos aspectos del driver.

Handler para manejo del GPTimer

`gptimer_handle_t`

Funcion para crear el GPTimer

`esp_err_t gptimer_new_timer(const gptimer_config_t *config, gptimer_handle_t *ret_timer)`

---> Parametros

- config: Puntero a la estructura de configuracion GPT [in]
- ret_timer: Puntero al manejador de GPT [out]

---> Retorna

- ESP_OK: GPTimer creado correctamente
- ESP_ERR_INVALID_ARG: GPTimer creado falló debido a un argumento no válido
- ESP_ERR_NO_MEM: GPTimer creado falló debido a falta de memoria
- ESP_ERR_NOT_FOUND: GPTimer creado falló debido a que todos los temporizadores de hardware se agotaron y no hay ninguno libre
- ESP_FAIL: GPTimer creado falló debido a otro error

Funcion para habilitar el GPTimer

`esp_err_t gptimer_enable(gptimer_handle_t timer)`

La funcion tambien incluye su respectiva funcion para desactivarla

---> Parametros

- timer: Manejador del GPTimer

---> Retorna

- ESP_OK: GPTimer habilitado sin errores
- ESP_ERR_INVALID_ARG: Habilitar GPTimer falló debido a un error de argumentos
- ESP_ERR_INVALID_STATE: Habilitar GPTimer falló debido a que ya estaba habilitado
- ESP_FAIL: Habilitar GPTimer falló debido a otros errores

Funcion para iniciar el contador

```
esp_err_t gptimer_start(gptimer_handle_t timer)
```

Su funcion inversa, para frenar el contador, es `gptimer_stop()`

---> **Parametros**

- timer: Manejador del GPTimer

---> **Retorna**

- ESP_OK: Inicio exitoso del timer
- ESP_ERR_INVALID_ARG: Inicio el contador fallo por error de armgumetos
- ESP_ERR_INVALID_STATE: Iniciar el contador fallo porque no estaba habilitado o ya se inicio
- ESP_FAIL: Iniciar el contador fallo debido a otro error

Funcion setear un valor al contador

```
esp_err_t gptimer_set_raw_count(gptimer_handle_t timer, uint64_t value)
```

---> **Parametros**

- timer: Manejador del GPTimer
- value: Valor a setear

---> **Retorna**

- ESP_OK: Valor seteado correctamente
- ESP_ERR_INVALID_ARG: Setar el valor fallo debido a error de argumentos
- ESP_FAIL: Setear el valor fallo debido a otros errores

Funcion para obtener el valor del contador

```
esp_err_t gptimer_get_raw_count(gptimer_handle_t timer, uint64_t *value)
```

Debido a que el valor que te devuelve la funcion es el crudo (raw), se suele utilizar esta funcion que te devuelve el valor calibrado, notese que el valor de retorno esta en Hz. `gptimer_get_resolution()`. Aunque la resolucion suele ser la misma que se seteo en la primera configuracion, ciertos relojes inestables hacen una calibracion. Para saber cuantos tiempo paso se utiliza el siguiente calculo: **time = value / resolution_hz**

---> **Parametros**

- timer: Manejador del GPTimer
- value: Puntero donde se devuelve el valor del contador

---> **Retorna**

- ESP_OK: Valor obtenido correctamente
- ESP_ERR_INVALID_ARG: Obtener el valor fallo debido a error de argumentos
- ESP_FAIL: Obtener el valor fallo debido a otros errores

Estructura para la configuracion de alarmas

`gptimer_alarm_config_t`

Variable	Descripcion	Tipo de variable
alarm_count	Valor objetivo de la alarma	uint64_t
reload_count	Valor de la alarma cuando se reinicio	uint64_t
auto_reload_on_alarm	Reinicia el valor del contador	uint32_t
flags	Flags de la estructura	int

---> ==Caracteristicas==

1. Cuando el contador llegue a este número, se dispara la alarma. Esta en Ticks, por lo tanto, hacer la conversion
2. Si el auto-reload está activo, el contador salta a este valor tras la alarma. Normalmente para empezar devuelta
3. Repeticion automatica. 1 para si, 0 para no

Funcion para setear la accion de la alarma

`esp_err_t gptimer_set_alarm_action(gptimer_handle_t timer, const gptimer_alarm_config_t *config)`

Debido a que la funcion se linkea en la memoria ram, la estructura de alarma debe estar como variable global o estatica.

---> Parametros

- timer: Manejador del GPTimer
- config: Puntero a la estructura con la configuracion de la alarma. Configurar con NULL significa desactivar la alarma

---> Retorna

- ESP_OK: Alarma configurada correctamente
- ESP_ERR_INVALID_ARG: Setear la alarma fallo debido a error de argumentos
- ESP_FAIL: El seteo fallo debido a otros errores

Estructura para registrar funciones en callbacks

`gptimer_event_callbacks_t`

Su unico miembro es la variable `on_alarm` y alli se dejan las funciones callbacks. El prototipo de estas funciones deben ser:

- Tipo de funcion: `bool`
- Ademas deben ser del tipo `IRAM_ATTR` para colocar la funcion en la ram en vez de que este en area de codigo

- Parametro 1: `gptimer_handle_t timer` que es el manejador del GPTimer
- Parametro 2: `const gptimer_alarm_event_data_t *edata` que es el dato de la alarma
- Parametro 3: `void *user_ctx` que es el dato del usuario, los parametros que recibe

Funcion para registrar un evento callback

```
esp_err_t gptimer_register_event_callbacks(gptimer_handle_t timer, const
gptimer_event_callbacks_t *cbs, void *user_data)
```

---> Parametros

- timer: Manejador del GPTimer
- cbs: Puntero a la estructura con las funciones
- user_data: Puntero en donde se envia los argumentos de las funciones

---> Retorna

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_ERR_INVALID_STATE: Set event callbacks failed because the timer is not in init state
- ESP_FAIL: Set event callbacks failed because of other error

Power modes

Libreria: `#include "esp_sleep.h"`

Tambien se suele necesitar `#include "driver/rtc_io.h"` que se usa para despertar al chip en algunos modos

Funcion para despertarse a partir de un determinado tiempo

```
esp_err_t esp_sleep_enable_timer_wakeup(uint64_t time_in_us)
```

---> Parametros

- time_in_us: Tiempo antes de levantarse, en microsegundos

---> Retorna

- ESP_OK: Si salio todo bien
- ESP_ERR_INVALID_ARG: Valor fuera de rango

Funcion para despertarse a partir de un pin

```
esp_err_t esp_sleep_enable_ext0_wakeup(gpio_num_t gpio_num, int level)
```

---> Parametros

- gpio_num: Numero de GPIO usado para levantarse. Solo los pines con RTC pueden ser usados. Los pines son: 0, 2, 4, 12-15, 25-27, 32-39
- level: Nivel de entrada en la que se despierta

---> Retorna

- ESP_OK: Si salio todo bien
- ESP_ERR_INVALID_ARG: Si el GPIO seleccionado no es RTC o modo invalido
- ESP_ERR_INVALID_STATE: Conflicto al disparar el despertador

Funcion para despertarse a partir de multiples pines

```
esp_err_t esp_sleep_enable_ext1_wakeup_io(uint64_t io_mask, esp_sleep_ext1_wakeup_mode_t level_mode)
```

---> Parametros

- io_mask: Mascara de bits de los GPIO que causan que se levanten. Aqui se ocupa lo mismo que se usa en la primera parte, la mascara de bits `1ULL << PinDeseado`. Si se quiere añadir mas, unirlos con OR `|`
- level_mode: Nivel de entrada en el que se despierta. Se usa `ESP_EXT1_WAKEUP_ALL_LOW` que despierta si TODOS los pines de la máscara están en bajo y `ESP_EXT1_WAKEUP_ANY_HIGH` que sespierta si CUALQUIERA de los pines de la máscara está en alto

---> Retorna

- ESP_OK: Si salio todo bien
- ESP_ERR_INVALID_ARG: Si los GPIO seleccionados no son RTC o modo invalido
- ESP_ERR_NOT_ALLOWED: Cuando el nivel de activación será diferente entre IO ext1 si `!SOC_PM_SUPPORT_EXT1_WAKEUP_MODE_PER_PIN`. Significa que si el chip no soporta configuración por pin (como el ESP32 original), dará error `ESP_ERR_NOT_ALLOWED` si intentas una configuración lógica imposible o mixta que el hardware no entienda.

Funcion para dormir, modo light

```
esp_err_t esp_light_sleep_start(void)
```

---> Retorna

- ESP_OK: Salio todo bien, retorna luego de despertarse
- ESP_ERR_SLEEP_REJECT: Modo dormir rechazado, fuente de activación establecida antes de la solicitud de suspensión
- ESP_ERR_SLEEP_TOO_SHORT_SLEEP_DURATION: Después de deducir la sobrecarga del flujo de sueño, la duración final del sueño es demasiado corta para cubrir la duración mínima del sueño del chip, cuando la fuente de activación del temporizador RTC está habilitada.

Funcion para dormir, modo deep

```
void esp_deep_sleep_start(void)
```

La fucnion no retorna debido a que cuando se levante, este empieza al inico del `app_main`, a diferencia de light sleep que sigue donde estaba

Funcion para saber la causa por la cual se levanto

```
esp_sleep_wakeup_cause_t esp_sleep_get_wakeup_cause(void)
```

La función retorna el motivo por el cual se despertó, se verifica con el tipo `esp_sleep_source_t` que pueden llegar a ser: por GPIO, touch, UART, wifi, entre otros.

Logging y Delays

Librería logging: `#include "esp_log.h"`

Macros utilizadas

- Error: `ESP_LOGE(TAG, "Error: %s", msg)` su color en consola es ROJO
- Advertencia: `ESP_LOGW(TAG, "Warning...")` su color en consola es AMARILLO
- Información general: `ESP_LOGI(TAG, "Info...")` su color en consola es VERDE
- Debugging: `ESP_LOGD(TAG, "Debug...")` su color en consola es BLANCO

EL TAG al inicio es un string, por ejemplo: `static const char *TAG = "MiProyecto"`

Librería delay: `#include "freertos/FreeRTOS.h" y #include "freertos/task.h"`

Función para esperar o delay

```
void vTaskDelay(const TickType_t xTicksToDelay)
```

---> Parametro

- `xTicksToDelay`: Cantidad de ticks que debe hacer delay, para pasar de milisegundos a ticks, usar la macro `pdMS_TO_TICKS(time_in_ms)`